DATABASE PRINCIPLES

CSC6302 · Fall 2026 - 2027

Module One Practice: Getting Started With Database Design

SUBMITTED BY

Keown "KeKe" White

Graduate Student, Computer Science

COURSE DETAILS

Professor Wendy-Beth Minton

Merrimack College

Submitted **OCT 31, 2026**

AI DISCLOSURE \\ PERSONAL DISCLOSURE

RESPONSE

ChatGPT was used as a tutoring resource to clarify database concepts, discuss SQL and Java syntax, and assist with sample test data. All submitted work was created, reviewed, tested, and understood by me.

Some of the mathematical notation, formal analysis, and technical methods used in this exercise come from material I have already encountered through previous computer science coursework and professional experience. This includes concepts learned across courses such as CSC6304, CSC6013, CSC6023, and other related classes, along with my bachelor's-level computer science background and earlier software engineering experience. Because of that background, I may use methods, notation, or verification techniques that go beyond what is directly required in the current assignment. These additions reflect concepts I already use when working through technical problems rather than material introduced only for this exercise.

Optional Extension: Paranormal Expedition Mode

The optional section does not need to be graded and is not part of the required exercise. I built it simply because I wanted to have some fun with the material and see how far I could extend the database beyond the assignment requirements, without changing the database entirely and creating new relations to select players and the infected.

The game reuses the same family vacation data and builds a small interactive application on top of it using Java, JDBC, MySQL, stored procedures, and a browser-based interface. It was created as an extra experiment in applying the course concepts in a more practical and creative way.

Personal Disclosure

Language and Spelling Note

My writing may occasionally switch between American English and British English spelling or usage. I completed my undergraduate education in the Caribbean, where British English conventions were commonly used, while my later graduate studies have been completed in the United States using American English. Because of that background, both forms sometimes appear naturally in my academic writing.

Problem

You may work on these problem sets together (citing your collaborators), but the write up must be your own. Assignments must be submitted digitally by the beginning of class; late assignments will receive a grade of zero (0). Your problem set must be updated to Canvas. You may update as many files as you wish to Canvas for the assignment. As I have a MAC, please do not submit Microsoft Word files. Export your file to a PDF

Design a database schema that keeps track of what you need to pack to go on vacation with your family to your favourite spooky destination. Consider data like:

- Where does each member of the family want to go?
- What activities could be done in those places?
- What costumes do you need to pack?
- What comfort/safety items do you need to pack? Hats? Gloves? Scarves? Warm wool socks? Boots? Snacks? Hand warmers? Glow sticks?

The goal here is to design a small database, and think about how to organise data. There is no right answer here, and as the course progresses, we will be talking about How to design efficient and well performing databases. For now, we are experimenting. Feel free to consider the above prompts, or come up with your own spooky destination database.

1. (100 Points) For your database design, you must:
    - (30 Points) List out the schema definition for at least three relations.
    - (10 Points) Select appropriate types for each column, and include them in your schema definitions.
    - (60 Points) List all the super keys, the candidate keys and the most reasonable primary key of each relation. Please ensure you justify your primary key choice.

Brief Database Purpose & Requirements

RESPONSE

The Family Spooky Vacation Database was designed to organise the information needed to plan a family trip involving several themed destinations. It records each member, the locations they prefer, activities available at those sites, and supplies required for travel. Packing information is separated into categories such as costumes, comfort items, safety equipment, food, and other essentials so individual needs can be assigned without duplicating the underlying item details. The structure also supports ranked destination choices and many-to-many relationships between people, places, activities, and personal packing lists. This design provides a clear way to answer practical planning questions while keeping related facts separated into appropriate relations.

Development Environment and Database Requirements

The required exercise was developed and tested on a Windows 11 system using the following environment:

- **Operating System:** Windows 11
- **Database Server:** MySQL Server 8.0.46
- **Database Administration Tool:** MySQL Workbench
- **Code Editor:** Visual Studio Code
- **Version Control:** Git
- **Database Schema:** `FamilySpookyVacation`

The submitted document can be reviewed without installing any additional software. MySQL is only needed if the database implementation is being recreated or tested.

To reproduce the relational design, MySQL Server 8.0 or a compatible release should be available. MySQL Workbench was used for schema administration, query execution, constraint testing, and EER diagram generation, although another MySQL-compatible client could perform the same database operations.

The implementation files are organised so the schema can be recreated from the supplied SQL scripts. The core database should be loaded in the following order:

1. `schema.sql`
2. `seed.sql`
3. `queries.sql`

`tests.sql` contains the additional checks used to verify constraints and referential behaviour. It is not required simply to create the database.

Relation Schemas and Key Analysis

RESPONSE

The database contains seven relations that separate family members, destinations, activities, packing items, and the associations connecting those subjects. Each definition includes the attribute data types, identified superkeys, candidate keys, selected primary key, and a brief justification for that selection.

1. FAMILY_MEMBER

FAMILY_MEMBER(MemberID INT UNSIGNED, MemberName VARCHAR(80))

Superkeys

- `{MemberID}`
- `{MemberID, MemberName}`

Candidate Key

- `{MemberID}`

Primary Key

- `MemberID`

Justification

`MemberID` provides a stable identifier for each relative without depending on a person's name for uniqueness. Two individuals could share the same name, while the assigned numeric value continues to distinguish their records.

2. VACATION_SITE

VACATION_SITE(SiteID INT UNSIGNED, SiteName VARCHAR(100),

Location VARCHAR(100), SpookinessLevel TINYINT UNSIGNED)

Superkeys

- `{SiteID}`
- `{SiteID, SiteName}`
- `{SiteID, Location}`
- `{SiteID, SpookinessLevel}`
- `{SiteID, SiteName, Location}`

- {SiteID, SiteName, SpookinessLevel}
- {SiteID, Location, SpookinessLevel}
- {SiteID, SiteName, Location, SpookinessLevel}

Candidate Key

- {SiteID}

Primary Key

- SiteID

Justification

SiteID uniquely identifies a destination without assuming that its descriptive information will remain unchanged or exclusive. A location or title may be modified later, whereas the identifier can continue representing the same place.

3. ACTIVITY

ACTIVITY(ActivityID INT UNSIGNED, ActivityName VARCHAR(100), ActivityCategory VARCHAR(40))

Superkeys

- {ActivityID}
- {ActivityID, ActivityName}
- {ActivityID, ActivityCategory}
- {ActivityID, ActivityName, ActivityCategory}

Candidate Key

- {ActivityID}

Primary Key

- ActivityID

Justification

ActivityID distinguishes each available experience independently of its description or category. Similar activities may appear under comparable names, making the numeric identifier a more dependable choice for establishing identity.

4. PACKING_ITEM

PACKING_ITEM(ItemID INT UNSIGNED, ItemName VARCHAR(100), ItemCategory VARCHAR(20))

Superkeys

- `{ItemID}`
- `{ItemID, ItemName}`
- `{ItemID, ItemCategory}`
- `{ItemID, ItemName, ItemCategory}`

Candidate Key

- `{ItemID}`

Primary Key

- `ItemID`

Justification

`ItemID` identifies one packing-item definition without tying its identity to descriptive wording. This approach allows an item's name or category to be adjusted without changing the value used by related records.

5. FAMILY_SITE

FAMILY_SITE(MemberID INT UNSIGNED, SiteID INT UNSIGNED, PreferenceRank TINYINT UNSIGNED)

Superkeys

- `{MemberID, SiteID}`
- `{MemberID, PreferenceRank}`
- `{MemberID, SiteID, PreferenceRank}`

Candidate Key

- `{MemberID, SiteID}`
- `{MemberID, PreferenceRank}`

Primary Key

- `{MemberID, SiteID}`

Justification

The combination of `MemberID` and `SiteID` represents one person's preference for one destination. Although `{MemberID, PreferenceRank}` also provides uniqueness under the stated business rule, a ranking may change as plans develop, while the relationship between the relative and location remains the same.

6. SITE_ACTIVITY

SITE_ACTIVITY(SiteID INT UNSIGNED, ActivityID INT UNSIGNED)

Superkeys

- `{SiteID, ActivityID}`

Candidate Key

- `{SiteID, ActivityID}`

Primary Key

- `{SiteID, ActivityID}`

Justification

Neither attribute can identify a row independently because one destination may provide several activities, while the same experience can be available at multiple locations. Their combination therefore represents one specific site-activity association without requiring an additional identifier.

7. MEMBER_ITEM

MEMBER_ITEM(MemberID INT UNSIGNED, ItemID INT UNSIGNED,

Quantity TINYINT UNSIGNED, PackedStatus BOOLEAN)

Superkeys

- `{MemberID, ItemID}`
- `{MemberID, ItemID, Quantity}`
- `{MemberID, ItemID, PackedStatus}`
- `{MemberID, ItemID, Quantity, PackedStatus}`

Candidate Key

- `{MemberID, ItemID}`

Primary Key

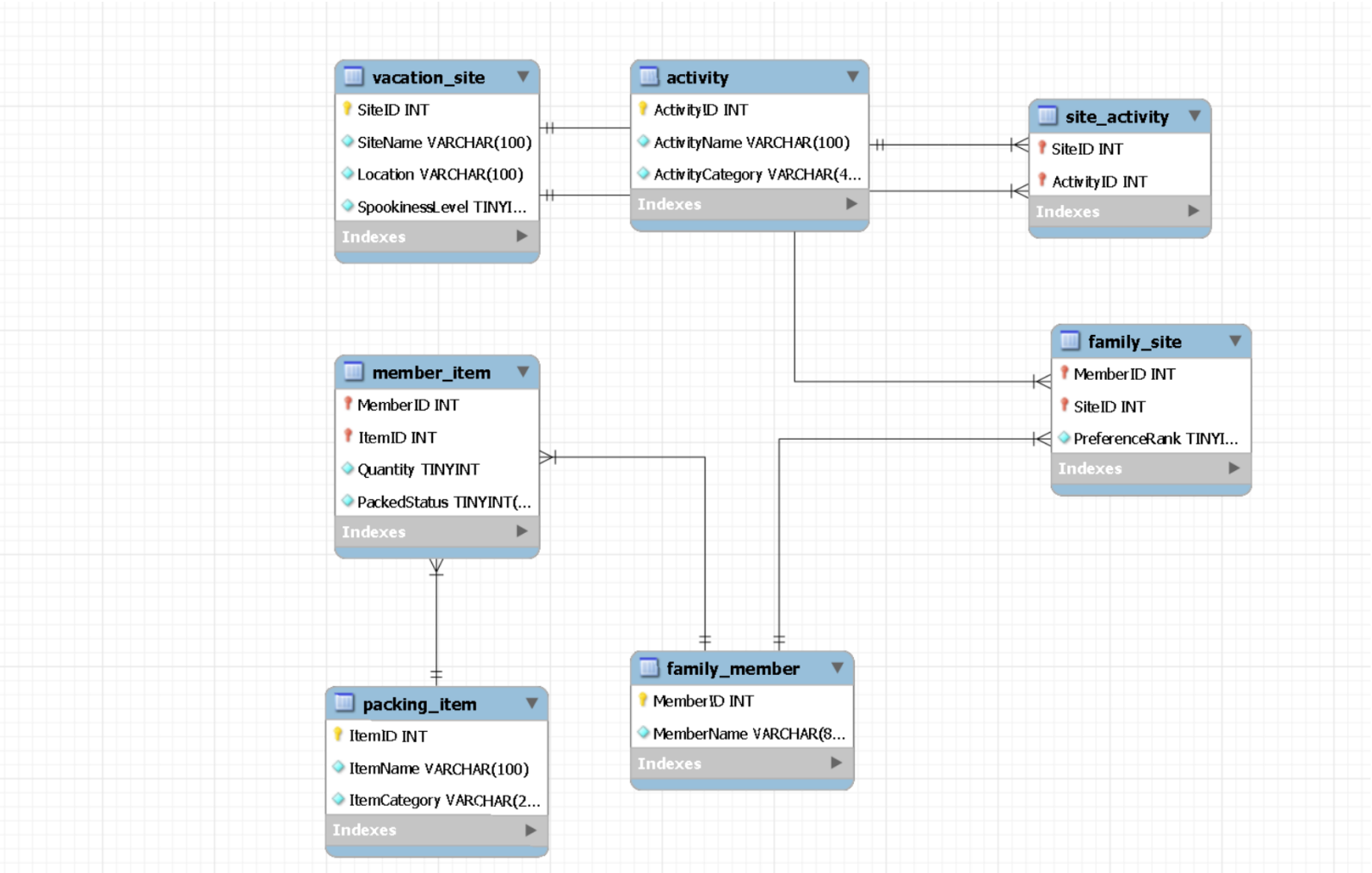
- `{MemberID, ItemID}`

Justification

The composite key identifies one packing item assigned to one family member. `Quantity` and `PackedStatus` describe that assignment rather than establish its identity, so neither attribute belongs in the minimal key.

EER DIAGRAM

RESPONSE



Design Summary and Mathematical Verification

RESPONSE

The relational structure was checked mathematically in addition to being tested through MySQL. Attribute closure was used to confirm candidate keys, set relationships were applied to verify superkeys, and functional dependencies were examined against the normal-form requirements. This provides a second check that the implemented schema follows the intended business rules.

Candidate-Key Verification

For any relation R with a proposed candidate key K:

$$K^+ = R$$

A candidate key must also be minimal:

$$\forall A \in K, (K - \{A\})^+ \neq R$$

Therefore, K must determine every attribute in R while no proper subset of K can do the same.

FAMILY_MEMBER

R = {MemberID, MemberName}

Functional dependency:

MemberID → MemberName

Attribute closure:

$$\{MemberID\}^+ = \{MemberID, MemberName\} = R$$

Because:

$$\emptyset^+ \neq R$$

the key cannot be reduced further.

Therefore:

Candidate Key = {MemberID}

VACATION_SITE

R = {SiteID, SiteName, Location, SpookinessLevel}

Functional dependency:

SiteID → SiteName, Location, SpookinessLevel

Attribute closure:

$\{SiteID\}^+ = \{SiteID, SiteName, Location, SpookinessLevel\} = R$

Since:

$\emptyset^+ \neq R$

`SiteID` is minimal.

Therefore:

Candidate Key = {SiteID}

ACTIVITY

R = {ActivityID, ActivityName, ActivityCategory}

Functional dependency:

ActivityID → ActivityName, ActivityCategory

Attribute closure:

$\{ActivityID\}^+ = \{ActivityID, ActivityName, ActivityCategory\} = R$

Since no smaller attribute set determines R:

Candidate Key = {ActivityID}

PACKING_ITEM

R = {ItemID, ItemName, ItemCategory}

Functional dependency:

ItemID → ItemName, ItemCategory

Attribute closure:

$\{ItemID\}^+ = \{ItemID, ItemName, ItemCategory\} = R$

Therefore:

Candidate Key = {ItemID}

FAMILY_SITE

R = {MemberID, SiteID, PreferenceRank}

The principal dependency is:

{MemberID, SiteID} → PreferenceRank

Under the business rule that one family member cannot assign the same preference rank to two destinations:

{MemberID, PreferenceRank} → SiteID

First closure:

$\{MemberID, SiteID\}^+ = \{MemberID, SiteID, PreferenceRank\} = R$

Second closure:

$\{MemberID, PreferenceRank\}^+ = \{MemberID, PreferenceRank, SiteID\} = R$

Minimality of the first set:

$\{MemberID\}^+ \neq R$

$\{SiteID\}^+ \neq R$

Minimality of the second set:

$\{MemberID\}^+ \neq R$

$\{PreferenceRank\}^+ \neq R$

Therefore:

$K_1 = \{MemberID, SiteID\}$

K_2 = {MemberID, PreferenceRank}

Both are candidate keys.

The selected primary key is:

PK = {MemberID, SiteID}

because `PreferenceRank` may change without altering the underlying member-destination relationship.

SITE_ACTIVITY

R = {SiteID, ActivityID}

No single attribute uniquely determines the relationship:

$\{SiteID\}^+ \neq R$

$\{ActivityID\}^+ \neq R$

Together:

$\{SiteID, ActivityID\}^+ = \{SiteID, ActivityID\} = R$

Therefore:

Candidate Key = {SiteID, ActivityID}

MEMBER_ITEM

R = {MemberID, ItemID, Quantity, PackedStatus}

Functional dependency:

{MemberID, ItemID} → Quantity, PackedStatus

Attribute closure:

$\{MemberID, ItemID\}^+ = \{MemberID, ItemID, Quantity, PackedStatus\} = R$

Neither component is sufficient by itself:

$\{MemberID\}^+ \neq R$

$\{ItemID\}^+ \neq R$

Therefore:

Candidate Key = {MemberID, ItemID}

Superkey Verification

A superkey S must contain a candidate key K: $K \subseteq S \subseteq R$

When a relation contains n attributes and has one candidate key containing k attributes, the number of supersets containing that key is: $N = 2^{(n-k)}$

This expression counts the candidate key itself plus every possible addition of the remaining n − k attributes.

FAMILY_MEMBER

n = 2
k = 1

$N = 2^{(2-1)}$
$N = 2^{1}$
N = 2

Superkeys:

{MemberID}

{MemberID, MemberName}

VACATION_SITE

n = 4
k = 1

$N = 2^{(4-1)}$
$N = 2^{3}$
N = 8

Therefore, eight superkeys contain `SiteID`.

ACTIVITY

n = 3
k = 1

$N = 2^{(3-1)}$
$N = 2^{2}$
N = 4

Therefore:

N = 4 superkeys

PACKING_ITEM

n = 3
k = 1

$N = 2^{(3-1)}$
$N = 2^2$
N = 4

Therefore:

N = 4 superkeys

SITE_ACTIVITY

n = 2
k = 2

$N = 2^{(2-2)}$
$N = 2^0$
N = 1

The only superkey is:

{SiteID, ActivityID}

MEMBER_ITEM

n = 4
k = 2

$N = 2^{(4-2)}$
$N = 2^2$
N = 4

The four superkeys are:

{MemberID, ItemID}

{MemberID, ItemID, Quantity}

{MemberID, ItemID, PackedStatus}

{MemberID, ItemID, Quantity, PackedStatus}

FAMILY_SITE With Two Candidate Keys

`FAMILY_SITE` requires a different calculation because two candidate keys exist:

K_1 = {MemberID, SiteID}

K_2 = {MemberID, PreferenceRank}

Let S(K) represent all superkeys containing candidate key K.

Then:

$|S(K_1)| = 2^{(3-2)} = 2$

$|S(K_2)| = 2^{(3-2)} = 2$

Both groups contain the full three-attribute set, so that overlap must only be counted once:

$|S(K_1) \cap S(K_2)| = 2^{(3-3)} = 1$

Using inclusion-exclusion:

$|S(K_1) \cup S(K_2)| = |S(K_1)| + |S(K_2)| - |S(K_1) \cap S(K_2)| = 2 + 2 - 1 = 3$

Therefore, the distinct superkeys are:

{MemberID, SiteID}

{MemberID, PreferenceRank}

{MemberID, SiteID, PreferenceRank}

Normalisation Verification

For a functional dependency:

$X \rightarrow Y$

Boyce-Codd Normal Form requires:

X is a superkey

for every non-trivial dependency in the relation.

Equivalently:

$\forall(X \rightarrow Y), Y \not\subset X \Rightarrow X$ is a superkey

For the principal relations:

MemberID → MemberName

SiteID → SiteName, Location, SpookinessLevel

ActivityID → ActivityName, ActivityCategory

ItemID → ItemName, ItemCategory

{MemberID, SiteID} → PreferenceRank

{MemberID, PreferenceRank} → SiteID

{MemberID, ItemID} → Quantity, PackedStatus

Each determinant is a candidate key or superkey of its relation.

Therefore:

∀ nontrivial X → Y, X is a superkey

∴ FAMILY_MEMBER ∈ BCNF

∴ VACATION_SITE ∈ BCNF

∴ ACTIVITY ∈ BCNF

∴ PACKING_ITEM ∈ BCNF

∴ FAMILY_SITE ∈ BCNF

∴ SITE_ACTIVITY ∈ BCNF

∴ MEMBER_ITEM ∈ BCNF

Since:

BCNF ⇒ 3NF ⇒ 2NF ⇒ 1NF

the design also satisfies the lower normal forms under the stated functional dependencies and business rules.

Referential and Implementation Verification

The relational model was also checked through executable constraints in MySQL. Primary-key duplication, invalid foreign-key references, missing required values, prohibited categories, repeated preference rankings, and non positive quantities were deliberately tested.

The verification can therefore be represented as two complementary checks:

$$V = M \land I$$

where:

M = mathematical verification

I = implementation verification

Both conditions were satisfied:

M = TRUE

I = TRUE

Therefore:

$$V = TRUE \land TRUE$$

$$V = TRUE$$

The schema is consequently supported by both relational analysis and observed database behaviour rather than relying on either form of verification alone.

END OF REQUIRED EXERCISE

The required portion of this assignment ends here.

The remaining pages are included only as an optional extension and do not need to be graded. I created the additional material to experiment with the database beyond the stated requirements and to have some fun applying the concepts in a working application.

Optional: Paranormal Expedition Mode

The Paranormal Expedition Mode was built as a separate application layer on top of the Family Spooky Vacation Database. It reuses the original members, destinations, and activities, then adds game-specific records for sessions, player status, contacts, observations, scoring, accusations, and fictional infection states.

One participant is secretly assigned as Patient Zero at the beginning of a session. The remaining players move through the expedition by selecting locations, completing available activities, recording interactions, and investigating clues. Evidence appears through fictional symptoms that become visible only when the correct conditions are met, which requires the group to compare timing, contact history, discovered information, and movement between sites before making an accusation.

The application uses Java with JDBC to communicate with MySQL, while stored procedures control the main game actions and enforce the rules defined in the database. A browser interface provides the player-facing controls without exposing hidden information such as Patient Zero or internal infection states.

This extension was created as a practical experiment rather than part of the required exercise. I spend a significant amount of my free time gaming and working with game-design ideas, so applying technical concepts through that perspective helps me understand them more clearly. Building the database into a playable system gave me a familiar way to explore relationships, state changes, stored procedures, and application logic while keeping the original assignment structure intact.

Optional: How the Original Database Is Reused

The game does not replace the original Family Spooky Vacation Database. Instead, it uses the existing relations as its foundation and adds a separate layer for interactive behavior. `FAMILY_MEMBER` becomes the source of player identities, so names are not duplicated inside the extension. `VACATION_SITE` supplies the locations used during each expedition, while `ACTIVITY` provides the actions that can be performed at those places. The existing `SITE_ACTIVITY` relation also remains important because it limits each destination to experiences that are actually available there.

This approach preserves the original design and allows the application to build on information that already exists. The optional tables reference the core relations through foreign keys rather than creating separate copies of the same facts.

The relationship can be represented as:

FAMILY_MEMBER → GAME_PLAYER

VACATION_SITE → CONTACT_EVENT

VACATION_SITE → ACTIVITY_EVENT

ACTIVITY → ACTIVITY_EVENT

SITE_ACTIVITY → ACTIVITY_EVENT

As a result, the class database remains complete on its own, while Paranormal Expedition Mode depends on that structure for its players, destinations, and available activities.